Database Technology, Management and Security

Project Type 4 — Experimental / Benchmarking Project

Measuring the Impact of Index Types and Tuning Settings on Query Execution Plans in PostgreSQL and MariaDB

Submitted By

Individual Submission

Name	Student ID	Email
Md. Imtiaj Alam Sajin	26-94090-2	26-94090-2@student.aiub.edu

Submitted To

Dr. Ashraf Uddin

Assistant Professor

Department of Computer Science

American International University-Bangladesh

Code, raw measurements and analysis

`https://github.com/Imtiaj-Sajin/Reseach-on-Database-Architecture`

Submission Date: 22 August 2026

Declaration of Originality

I declare that this report is my own original work. All sources of information have been acknowledged and cited using the IEEE referencing style. No part of this report has been copied from any other student's work or from any other source except where explicitly cited. I understand that any act of plagiarism will be dealt with according to the university's academic integrity policy.

All measurements reported here were produced by software written for this project and published with it [1]. Every figure and table is generated directly from the raw measurement files; no numerical value in this report was transcribed by hand.

Repository. All source code, every raw measurement, the generated figures and an executable analysis notebook are publicly available at:

`https://github.com/Imtiaj-Sajin/Reseach-on-Database-Architecture`

No dataset is distributed. All data is generated from a fixed seed, so the corpus is byte-identical on any machine and every result in this report can be regenerated with a single command.

Md. Imtiaj Alam Sajin

Abstract

Choosing the wrong index can slow a query by orders of magnitude, and a large body of practitioner guidance exists to prevent that outcome. This study asks whether the guidance survives measurement. A controlled benchmark was built in which value skew, predicate correlation and physical clustering vary independently, with the true cardinality of every predicate computed from the generated data rather than obtained from the database, so that estimation error can be measured directly. 23,480 query executions were recorded against PostgreSQL 17.1 and MariaDB 10.4.28 across seven table scales, eleven dataset configurations, ten index configurations and sixteen optimiser settings.

The first result is reassuring: with a conventional index set PostgreSQL selects a near-optimal access path, erring materially for only 2.3% of queries. The remaining results are not. Three widely recommended practices each degrade performance under identifiable conditions. Extended statistics improve cardinality estimates by up to 108 times while making the affected queries up to 2.7 times slower. Lowering `random_page_cost` on solid-state storage slows the workload by 2.3 times, the default lying within 6% of optimal. Adding a BRIN index alongside a B-tree raises misselection from 2.3% to 73.3%, with individual queries degrading by up to 194 times, and the cause is located in a specific line of PostgreSQL's source code.

Cardinality misestimation, the explanation the literature would predict, does not account for these failures: no query in the experiment exhibits both severe misestimation and material regret. Measuring a second database separates general properties from implementation quirks, and shows the two systems degrade in opposite directions as tables grow.

Keywords: query optimisation; access path selection; index selection; cardinality estimation; cost model; PostgreSQL; MariaDB; performance benchmarking

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and Motivation

When a database executes a query, it must decide *how* to find the qualifying rows: read the table sequentially, or use one of the available indexes, and if so which one. This is the access-path decision. It was formalised in the System R optimiser [2] and the same cost-based framework, estimate the cardinality and then price each candidate path, underpins PostgreSQL and MariaDB today. Depending on that single decision, an identical query over identical data can differ by two orders of magnitude in execution time.

Because the decision is invisible to the user and consequential to performance, a large body of tuning guidance has accumulated around it. Administrators are advised to create extended statistics when predicate columns are correlated [3], to lower `random_page_cost` when running on solid-state storage [4], and to add Block Range Index (BRIN) structures on naturally ordered columns because they are small and inexpensive to build [5]. This guidance appears in official documentation, vendor engineering material and community forums.

Such guidance is almost always justified by a *mechanism argument*, an explanation of why the change ought to help, rather than by workload-level measurement. Mechanism arguments are persuasive because they are usually correct as far as they go. Extended statistics really do improve cardinality estimates. Random reads on flash storage really are cheaper relative to sequential reads than they were on magnetic disks [6], [7]. BRIN indexes really are orders of magnitude smaller than B-trees. The question this study asks is whether being correct about the named quantity is sufficient to improve the outcome that actually matters, namely the runtime of the workload.

1.2 Problem Statement

Prior work established that cardinality misestimation is the dominant source of poor query plans [8], [9], but studied join ordering rather than single-relation access-path selection. Whether the shipped optimiser selects access paths well, and whether the remedies commonly recommended for it actually improve matters, has not been examined systematically. This study addresses that gap through six research questions.

RQ1 How often does the optimiser select a slower access path than one available to it, and what does that cost?

RQ2 Which data properties drive misselection: value skew, correlation between predicate columns, or physical clustering?

RQ3 Is cardinality misestimation the cause, as the query optimisation literature would predict?

RQ4 Do the three tuning practices named above improve access-path selection?

RQ5 Under what conditions do the observed effects cease to hold?

RQ6 Which effects are properties of cost-based access-path selection in general, and which are artefacts of one implementation?

RQ5 deserves emphasis. Studies frequently report that an effect exists without establishing when it disappears, leaving a practitioner unable to judge whether it applies to their deployment. Section 4.6 shows this is not a hypothetical concern: a two-point comparison in this very study would have produced the wrong conclusion.

1.3 Objectives and Scope

The objective is to build a benchmark in which skew, predicate correlation and physical clustering vary independently with exact ground-truth cardinality; to quantify a modern optimiser's access-path selection quality; to evaluate three recommended tuning practices and explain any failures mechanistically rather than descriptively; to establish the conditions under which the observed effects cease to hold; to repeat the study on a second database so that general results can be separated from implementation-specific ones; and to publish a fully reproducible artifact. Section 1.4 states what was achieved against each.

Excluded from scope. Join ordering and join method selection, which are separate and well-studied problems [9]; cold-cache behaviour, because reliably evicting the operating system page cache is not portable across platforms; and multi-user concurrency, since all measurements are single-session.

1.4 Contributions

1. A controlled benchmark in which three data properties vary independently, with ground-truth cardinality obtained by counting the generated data rather than by querying the system under test.
2. A quantification of PostgreSQL 17's access-path selection quality, found to be near optimal under a conventional index set.
3. Evidence that cardinality misestimation does *not* explain single-relation access-path misselection, in contrast to the established result for join ordering.
4. Three measured cases in which recommended tuning practice degrades performance, each traced to a mechanism visible in the query plan, one of them to a specific line of source code.
5. A characterisation of the scale boundaries of the largest effect, showing that its frequency and its severity collapse at two different and separately identifiable system boundaries.
6. A cross-database comparison separating general properties from implementation artefacts, including the finding that the two systems degrade in opposite directions as data grows.

1.5 Report Organisation

Section 2 reviews the relevant theory and positions the work against prior literature. Section 3 describes the experimental design and measurement protocol. Section 4 presents the results. Section 5 interprets them, connects them to course concepts and states threats to validity. Section 6 concludes.

2 Background and Literature Review

2.1 Key Concepts

Access-path selection. Given a predicate over a single relation, a cost-based optimiser enumerates candidate access paths, a sequential scan plus one path per usable index, estimates the number of rows each will produce, and assigns each a cost [2], [10], [11]. In PostgreSQL, cost is expressed in abstract units anchored to `seq_page_cost`, with `random_page_cost` expressing how much more expensive a randomly fetched page is assumed to be relative to a sequentially fetched one [12]. The default ratio of four to one encodes an assumption about magnetic disk behaviour.

A *bitmap scan* occupies an intermediate position between an index scan and a sequential scan. The index is scanned to build a bitmap of matching tuple identifiers, the bitmap is sorted into physical order, and the heap is then read in that order, converting random access into something closer to sequential access. This intermediate turns out to matter a great deal in the cross-system comparison of Section 4.7.

Index structures. A B-tree supports both equality and range predicates and yields exact tuple pointers [13], [14]. A hash index supports equality only. A BRIN index stores summary information, typically minimum and maximum values, for each range of physical blocks rather than for each tuple [5]. It is consequently very small, often by three orders of magnitude, but it returns *candidate block ranges* rather than rows, so every tuple in a candidate range must be rechecked against the predicate. Its usefulness therefore depends entirely on the correlation between indexed value and physical row position. BRIN exemplifies the specialisation trend identified by Stonebraker and Cetintemel [15]: it abandons generality for a large size reduction, and is correspondingly effective only on the narrow class of workloads whose physical layout suits it.

Cardinality estimation. PostgreSQL estimates the selectivity of a conjunctive predicate by multiplying per-column selectivities, which assumes column independence [16]. Where columns are correlated the estimate can be wrong by orders of magnitude, and the error is systematically an underestimate for positively correlated attributes. Extended statistics objects, created with `CREATE STATISTICS`, capture multivariate distributional information and are the documented remedy [3].

Buffer management. PostgreSQL maintains its own buffer pool, sized by `shared_buffers`, above the operating system page cache [17]. A logical read satisfied from the buffer pool costs a pointer dereference; one that misses requires a system call and a memory copy even when the operating system holds the page. This layering matters here because a working set may exceed `shared_buffers` while remaining entirely within the page cache, producing a regime that is neither fully resident nor genuinely I/O bound.

2.2 Related Work

Optimiser quality and cardinality estimation. Leis et al. [8], [9] introduced the Join Order Benchmark and established that cardinality misestimation, rather than cost-model calibration or plan enumeration, dominates plan quality for join queries. Errors compound multiplicatively through join trees, as earlier theoretical work predicted [18]. Their focus is join ordering; the single-relation access-path decision is not isolated. The results reported here are complementary and, on one point, opposed: for the access-path decision misestimation is *not* the primary driver of the misselection observed (Section 4.2).

Moerkotte et al. [19] introduced q-error, the symmetric ratio between estimated and actual cardinality, and proved bounds relating it to plan cost degradation; it is adopted unchanged here. Han et al. [20] benchmark cardinality estimators comprehensively and likewise find estimation accuracy and plan quality imperfectly coupled; Negi et al. [21] make the same observation constructively, training estimators against a plan-cost objective rather than an accuracy objective.

Cost models and learned optimisers. Wu et al. [22] found optimiser cost models usable for predicting execution time only after calibration. Work replacing the analytic model with a learned one [23], [24] rests on a premise rarely quantified for a specific decision: that the shipped cost model is miscalibrated badly enough to be worth replacing. Where that literature asks how much a learned model could gain, this study asks how much the existing model actually loses. Lan et al. [25] note the scarcity of controlled studies isolating individual optimiser components.

Index selection and physical design. A long line of work addresses which indexes to create [26], recently evaluated comparatively by Kossmann et al. [27]. That literature asks which index set minimises cost, assuming the optimiser will use the chosen indexes appropriately. This study examines the complementary and apparently unexamined question of whether *adding* an index can degrade performance by changing which paths the optimiser considers, and answers it affirmatively in Section 4.5.

Benchmarking methodology. Wongkham et al. [28] model rigorous index evaluation, notably measuring each index design in isolation rather than inferring behaviour from composite configurations, a practice adopted here; their subject, learned indexes [29], differs but the discipline transfers. Raasveldt et al. [30] catalogue pitfalls in performance testing, including measurement instruments whose overhead depends on the plan being measured, which Section 3.8

describes avoiding. Boncz et al. [31] illustrate why a fixed dataset is unsuitable here: fixing the data fixes skew, correlation and clustering at whatever values it happens to have, making any observed effect impossible to attribute to a particular property.

2.3 Research Gap

Three gaps follow. First, the access-path decision has not been isolated and measured in the way join ordering has, so the extent to which the join-ordering conclusion transfers is unknown. Second, the tuning practices recommended for this decision rest on mechanism arguments rather than workload-level measurement. Third, published effects are seldom accompanied by the conditions under which they cease to hold, so a practitioner cannot determine applicability. This study addresses each.

3 Methodology

3.1 Overview of Approach

The study measures, for each query, the execution time of the plan the optimiser freely chose against the fastest plan obtainable by restricting the available indexes. The ratio between them isolates the quality of the optimiser's decision from the intrinsic difficulty of the query.

Because neither system provides a mechanism to hide one index from the optimiser while leaving others visible, each candidate access path is measured by building that index *alone* and executing the full query set, following Wongkham et al. [28]. A further configuration builds every index at once, reproducing a realistic deployment, and is where the optimiser's free choice is observed.

3.2 Evaluation Metrics

q-error [19] measures estimation accuracy:

$$q(\hat{n}, n) = \frac{\max(\hat{n}, n)}{\min(\hat{n}, n)}, \tag{1}$$

where $\hat{n}$ is the estimate and n the true row count, both floored at one. A value of 1.0 denotes a perfect estimate. The symmetric ratio form is used because a tenfold underestimate and a tenfold overestimate are comparably damaging to plan selection.

Access-path regret measures the cost of the decision itself:

$$R(q) = \frac{T_{\text{chosen}}(q)}{\min_{a \in A} T_a(q)}, \tag{2}$$

where A is the set of index configurations and T_a is median execution time. $R = 1$ indicates the

optimiser selected the best available path; $R = 4$ indicates the query took four times longer than necessary. Because the denominator is the fastest configuration *measured* rather than a proven optimum, reported regret is a **lower bound** on the true penalty.

Misselection is defined as $R > 1.2$. The threshold prevents run-to-run variation being counted as an optimiser error; it is a judgement call, and full distributions are published so a different one may be applied.

3.3 Controlled Data Generation

Real datasets do not permit varying one property while holding others constant, so all data is generated. Three factors vary independently around a uniform, independent baseline, so that each effect is attributable (Table 1). Eleven dataset configurations result. All randomness is seeded, so the corpus is byte-for-byte reproducible.

Table 1: Controlled factors. Each dataset varies a single factor from the baseline, so that any observed effect is attributable to that factor alone.

Factor	Levels	What it controls
`zipf_s`	0.0, 0.5, 1.0, 1.5	Value skew. At 0.0 the top ten values hold 0.3% of rows; at 1.5 they hold 76.8%.
`dep_strength`	0.0, 0.25, 0.5, 0.75, 1.0	Probability that column b is a deterministic function of column a, the structure extended statistics is designed to capture.
`physical_corr`	0.0, 0.5, 0.95, 1.0	Correlation between value order and physical row order, the property BRIN depends on entirely.

Domain sizing is load bearing. With 10^6 rows and 100 distinct values in each of columns a and b, an independent conjunction `a = va AND b = vb` returns approximately 100 rows, exactly what the independence assumption predicts, which serves as the control. Under a full functional dependency the same predicate returns all 10^4 rows sharing `a = va` while the estimate remains at 100. That is a q-error of 100 at one percent selectivity, precisely the region in which index and sequential access compete most closely.

3.4 The Datasets

Varying one factor at a time from the baseline yields the eleven datasets in Table 2. Every dataset shares the same schema, row count and generation seed; only the named factor differs. Each carries seven columns: an identifier, a uniform control column, the skewed column `k_skew` with 10,000 distinct values, the correlated pair a and b with 100 distinct values each, the physically clustered timestamp `ts`, and a 48-character payload so that rows have realistic width. Without the payload the table is so narrow that a sequential scan is almost always cheapest and the access-path decision becomes uninteresting.

Table 2: The eleven dataset configurations. Bold marks the single factor that differs from the baseline in each case.

Dataset	Varies	zipf_s	dep_strength	physical_corr	Purpose
baseline	—	0.00	0.00	0.00	control
skew05	skew	**0.50**	0.00	0.00	mild skew
skew10	skew	**1.00**	0.00	0.00	strong skew
skew15	skew	**1.50**	0.00	0.00	extreme skew
dep025	correlation	0.00	**0.25**	0.00	weak dependency
dep05	correlation	0.00	**0.50**	0.00	medium dependency
dep075	correlation	0.00	**0.75**	0.00	strong dependency
dep10	correlation	0.00	**1.00**	0.00	total dependency
phys05	clustering	0.00	0.00	**0.50**	BRIN's hard case
phys095	clustering	0.00	0.00	**0.95**	nearly sorted
phys10	clustering	0.00	0.00	**1.00**	BRIN's best case

3.5 Index Configurations

Each candidate access path is measured in isolation, then two configurations allow the optimiser to choose freely (Table 3). The isolated configurations exist solely to establish what the best achievable plan was, so that the gap can be measured; the free-choice configurations are the subject of study.

Table 3: The ten index configurations. MariaDB provides six of them: InnoDB offers neither BRIN nor hash indexes, so those have no counterpart and are omitted rather than approximated.

Configuration	Contains	Serves	MariaDB
none	no indexes	sequential-scan baseline	yes
btree_skew	B-tree on k_skew	equality, range	yes
hash_skew	hash on k_skew	equality only	—
brin_skew	BRIN on k_skew	equality, range	—
btree_ts	B-tree on ts	timestamp range	yes
brin_ts	BRIN on ts	timestamp range	—
btree_a_b_separate	two B-trees, a and b	conjunctions	yes
btree_ab_composite	one B-tree on (a, b)	conjunctions	yes
all	every index above	**optimiser chooses**	—
all_no_brin	every index except BRIN	**optimiser chooses**	yes

3.6 Scale of the Study

Combining eleven datasets, ten index configurations and 66 distinct queries per dataset, each executed seven times, and repeating the exercise across seven table sizes and two database systems, yields the matrix in Table 4. The intermediate sizes run only the three phys datasets,

because the boundary question of Section 4.6 concerns BRIN and BRIN depends only on physical clustering; running the other eight there would add hours and answer a question not asked.

Table 4: Complete experiment matrix. Each figure is a count of measured query executions, every one of which is published in the artifact [1].

Table size	Heap	Fits in 128 MB pool?	Datasets	PostgreSQL	MariaDB
1,000,000	112 MB	yes	11	2,398	1,232
1,250,000	140 MB	no	3	654	336
1,500,000	168 MB	no	3	654	336
2,000,000	223 MB	no	3	654	336
3,000,000	335 MB	no	3	654	336
5,000,000	558 MB	no	3	654	336
10,000,000	1,116 MB	no	11	2,398	1,232
Cost-model sweep	7 `random_page_cost` levels		5	6,230	—
Configuration sweep	9 optimiser settings		5	—	5,040
Total				**14,296**	**9,184**

The two scales run with all eleven datasets, 10^6 and 10^7 rows, were chosen to sit on opposite sides of the buffer pool: at one million rows the heap is 112 MB against a 128 MB pool and therefore resident, while at ten million it is 1,116 MB and cannot be. That contrast turns out to determine whether an entire finding holds (Section 4.6).

3.7 Query Families

Four families are used. Every predicate's true cardinality is computed by counting the generated arrays directly, never by asking the system under test:

- **Equality** on the skewed column; hash, B-tree, BRIN and sequential scan all compete.
- **Range** on the skewed column; hash cannot serve this, which tests fallback behaviour.
- **Range on the physically clustered column**, BRIN's most favourable case.
- **Conjunctive** on the correlated pair, in dependent and independent variants, the latter serving as a control.

Target selectivities swept are 0.001%, 0.01%, 0.1%, 1%, 5%, 10%, 20% and 50%. Predicate constants are chosen by a two-pointer sweep over the distinct values and their cumulative counts, selecting the contiguous value range whose total is nearest the target, and the *achieved* selectivity is always reported rather than the target.

3.8 Measurement Protocol

Every timed execution used `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)` on PostgreSQL and `ANALYZE FORMAT=JSON` on MariaDB, reading the reported execution time. Three considerations motivate this choice, each bearing on validity [30]:

1. The statement executes server-side and discards the result set, so client transfer never enters the measurement.
2. `TIMING OFF` disables per-node instrumentation. With timing enabled, PostgreSQL reads the system clock twice per emitted row. That overhead is *plan-dependent*: a sequential scan emitting 10^6 rows pays it a million times while an index scan emitting 10^4 rows pays it ten thousand times. Using the instrumented form would therefore bias the comparison toward plans emitting fewer rows, which is precisely the comparison being made.
3. Applying one instrument uniformly means any residual overhead is common mode and cancels in the ratio defining regret.

Each query executed seven times; the first was discarded as cache-warming and the median of the remaining six reported, with all individual runs retained so variance claims can be verified independently. Parallelism was disabled so it could not confound the serial access-path decision, and just-in-time compilation was disabled because it engages on cost thresholds and would fire inconsistently across configurations.

3.9 Measurement Rigor and Validity Checks

Nineteen automated checks execute before any measurement is trusted. Zipfian skew concentrates the top ten values from 0.3% of rows at $s = 0$ to 76.8% at $s = 1.5$. Realised dependency strength lands within 0.005 of the requested value at every level. Realised physical rank correlation is 0.005, 0.497, 0.949 and 1.000 for the four settings, and is *independently confirmed against PostgreSQL's own* `pg_stats` *correlation estimate* at run time, which validates the generator against a measurement outside our control. Every predicate's row count matches a direct recount of the generated arrays, and identical seeds produce identical data.

Three measurement errors were found and corrected during the study, and are reported rather than omitted. First, the range predicate builder originally anchored a fixed-width window at the median row position; on a Zipfian column the median row falls inside a single very frequent value, so the bounds collapsed and every target below 13.6% produced an identical query on the most skewed dataset. Affected measurements were discarded and recollected. Second, row counts mean different things in the two systems: PostgreSQL reports rows *after* the predicate is applied while MariaDB reports rows *examined* with the surviving fraction given separately, and comparing them directly would have made every cross-system q-error meaningless. Third, an estimate that appeared too accurate to be genuine was confirmed real by re-running through `EXPLAIN`, which does not execute the query and reported the same value.

3.10 Dispersion and Outlier Treatment

All six retained repetitions are stored per measurement, so the spread behind every reported median can be examined. Dispersion is quantified as the coefficient of variation (CV), the

standard deviation over the mean, which stays comparable across queries spanning four orders of magnitude in duration.

Table 5 shows it is concentrated almost entirely in sub-millisecond measurements. The median CV is 1.79% for PostgreSQL measurements at or above 1 ms and 24.47% below it, where timer granularity rather than the access path dominates. This independently justifies the 1 ms reliability floor applied throughout the analysis, which was fixed before this check was run. Above that floor the choice of central estimator is immaterial for PostgreSQL: mean and median differ by more than 10% in only 2.2% of cases.

Individual timings falling outside the Tukey fence of their own repetition group are retained rather than deleted. With six repetitions a fence is a weak instrument, and discarding runs would bias the result toward the expected outcome. They are addressed by construction instead, since a median is insensitive to up to two outlying runs in six. The residual risk sits with MariaDB, where 17.3% of analysed measurements exceed a 20% CV against 2.8% for PostgreSQL; MariaDB timing differences below roughly a factor of two are therefore not treated as meaningful anywhere in this report.

Table 5: Run-to-run dispersion over all retained repetitions. CV is the coefficient of variation; "flagged" is the share of individual timings falling outside the Tukey fence of their own repetition group.

System	Subset	n	Median CV	90th pct	Flagged
PostgreSQL	median $\geq$ 1 ms, analysed	8,138	1.79%	10.35%	6.76%
PostgreSQL	median $<$ 1 ms, excluded	6,158	24.47%	36.26%	10.41%
MariaDB	median $\geq$ 1 ms, analysed	3,033	0.84%	52.27%	8.36%
MariaDB	median $<$ 1 ms, excluded	1,223	43.39%	120.62%	11.11%

3.11 Experimental Environment

Both buffer pools are set to 128 MB deliberately. That quantity is what the scale-boundary result of Section 4.6 turns on, so the two systems must agree on it or the comparison is meaningless.

4 Results and Findings

4.1 RQ1: Baseline Optimiser Quality

Under a conventional index set, that is B-tree and hash indexes without BRIN, PostgreSQL performs well (Table 7). At 10^6 rows it selected the best available access path for 97.7% of queries, and where it erred the median penalty was 1.3%.

This positive result frames what follows: the failures reported below are specific problems rather than symptoms of a generally poor optimiser. It is a statement about PostgreSQL and not about cost-based optimisers as a class, as Section 4.7 demonstrates.

Table 6: Experimental environment. Complete settings are captured automatically per run in the artifact [1].

Component	Specification
CPU	Intel Core i5-12400, 6 cores / 12 threads
Memory	23.8 GB
Storage	SSD, dedicated tablespace on a device separate from the OS
Operating System	Windows 11 Pro (build 26220)
Primary DBMS	PostgreSQL 17.1, x86_64, MSVC 19.41
Second DBMS	MariaDB 10.4.28
Buffer pool	`shared_buffers` 128 MB; `innodb_buffer_pool_size` 128 MB
`work_mem`	4 MB
`random_page_cost`	4.0 (default), swept 4.0 to 1.0
Table scales	10^6 to 10^7 rows, seven scales
Total executions	23,480

Table 7: Access-path selection quality under a conventional index set, PostgreSQL.

Scale	Queries	Misselection	Median R	p90 R	Max R
10^6	130	2.3%	1.013	1.13	1.39
10^7	155	32.3%	1.043	1.52	3.70

4.2 RQ3: Misestimation Does Not Explain Misselection

Of the 113 misselected queries at 10^6 rows, the median q-error was 1.017 and 98.2% had q-error below 1.5. The optimiser's row estimates were essentially correct and it selected badly regardless (Figure 1).

Decomposing by query family makes the point far more strongly (Table 8). Severe misestimation is present in the workload, reaching a median q-error of 67.8 on conjunctive predicates, and the optimiser nonetheless selects the best available plan for *every one of them*. Meanwhile the range families, where estimates are accurate to within 2%, account for essentially all misselection.

Table 8: Estimation error and plan regret by query family, 10^6 rows. The two quantities occupy disjoint parts of the workload.

Family	Queries	Median q-error	Median regret	Misselection
`conj`	19	**67.81**	**0.99**	**0.0%**
`eq`	3	1.03	1.77	66.7%
`range`	55	1.01	3.03	98.2%
`ts_range`	53	1.02	2.30	66.0%

The separation is absolute rather than merely statistical. Every one of the 19 queries with q-error above 10 belongs to the `conj` family and none is misselected; every one of the 91 misselected queries has q-error below 10, with a median of 1.015. **No query in the experiment exhibits both severe misestimation and material regret.**

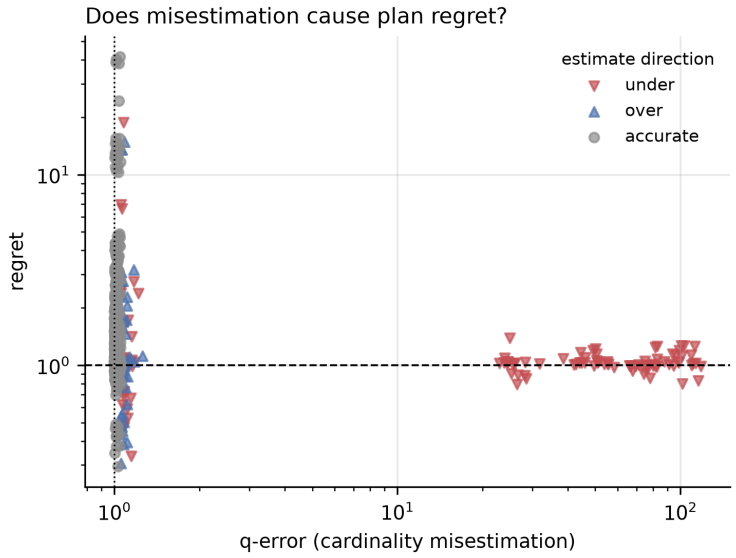

Figure 1: Access-path regret against cardinality estimation error, by direction of the error. High-regret queries cluster at q-error near one, so misestimation does not account for them.

The divergence from Leis et al. [8] is structural rather than contradictory. Join ordering has a plan space that grows combinatorially, and estimation errors propagate multiplicatively up the join tree [18], so a modest leaf error becomes a large root error. Single-relation access-path selection has neither property: the candidate set is three or four paths, the estimate enters the cost function once, and nothing amplifies it. What remains is the accuracy of the cost function itself and the completeness of path enumeration, which is exactly where the failures below land.

Figure 2 separates estimation error by source, confirming that the weakness is specific to conjunction across correlated columns rather than to selectivity estimation in general.

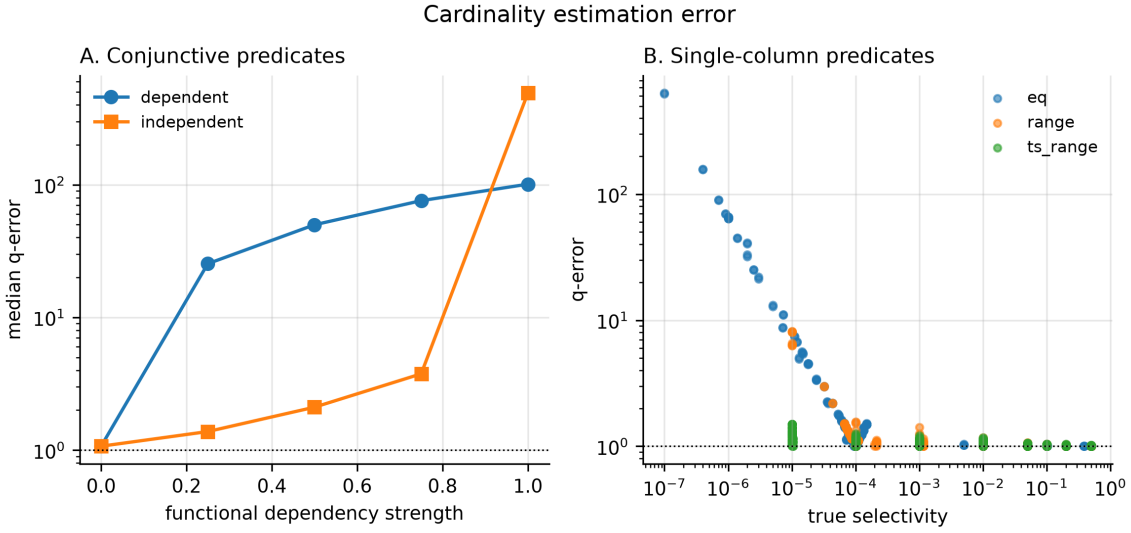

Figure 2: Cardinality estimation error by source. Panel A, conjunctive predicates against dependency strength, with the independent variant as control. Panel B, single-column predicates against selectivity.

4.3 RQ4a: Extended Statistics

`CREATE STATISTICS` is the documented remedy for correlated predicates [3]. It succeeds at its stated objective and degrades the outcome (Table 9, Figure 3).

Table 9: Effect of extended statistics on dependent conjunctive predicates, 10^6 rows. Estimates become nearly exact while every affected query becomes slower.

Dep.	q-**error**		**Est. rows**	**True**	**Median ms**		**Slow-**
strength	without	with	with	rows	without	with	**down**
0.25	25.23	1.11	2,767	2,556	0.93	2.54	**2.73**×
0.50	49.52	1.04	5,200	5,053	2.12	3.53	1.66×
0.75	75.89	1.02	7,633	7,513	2.80	4.34	1.55×
1.00	113.19	1.05	9,700	9,995	3.95	5.06	1.28×

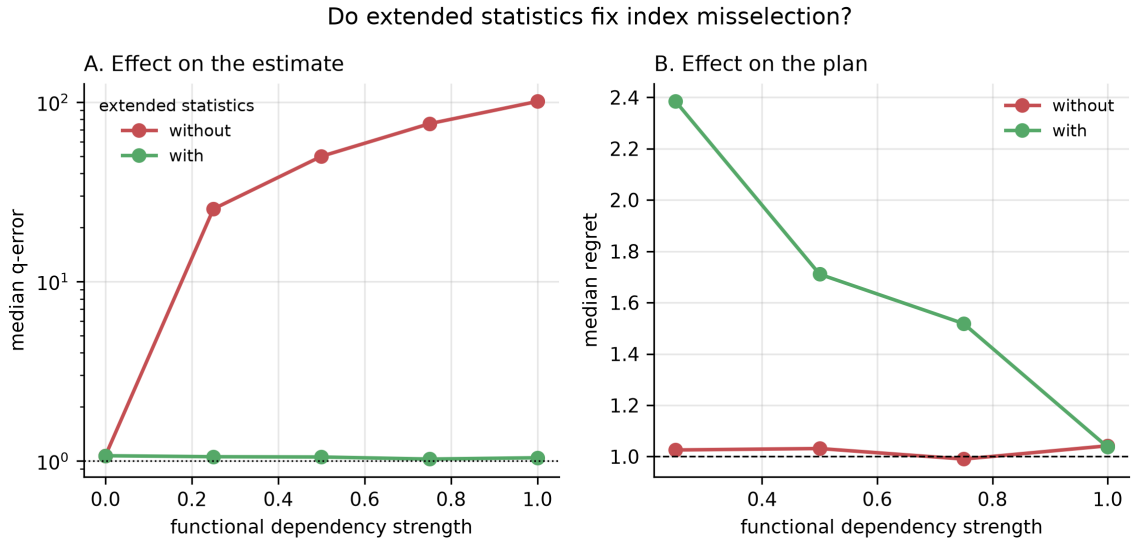

Figure 3: Extended statistics against dependency strength. Panel A, the estimate improves by up to two orders of magnitude. Panel B, the resulting plan degrades. Improving the estimate and improving the plan are separable outcomes.

Root cause. Without extended statistics the optimiser uses the composite index on (a, b) in a single index scan. With them it switches to a `BitmapAnd` over two single-column indexes. Both plans fetch the identical 7,317 heap blocks, but the second performs two index scans plus a bitmap intersection in place of one scan. The reason for the switch is visible in the node-level estimates:

```
Bitmap Heap Scan  ... rows=10367     <- extended statistics applied
  -> BitmapAnd    ... rows=107       <- independence assumption, NOT
     corrected
```

Listing 1: Node row estimates with extended statistics present.

Since $10367^2/10^6 \approx 107$, the `BitmapAnd` node is still multiplying per-column selectivities under exactly the independence assumption that extended statistics exist to replace. The optimiser therefore prices the `BitmapAnd` at 635 using the *uncorrected* figure and the composite path at 13,558 using the *corrected* one, a 21-fold discrepancy, and selects the plan that is in fact slower.

4.4 RQ4b: Lowering `random_page_cost`

The default `random_page_cost` of 4.0 encodes a magnetic-disk assumption, and guidance widely recommends reducing it on flash storage. Measured across the full query set, the recommendation is counterproductive (Table 10, Figure 4).

Table 10: Total execution time over the full query set at each `random_page_cost`. The default lies within 6% of optimal.

`random_page_cost`	**Total time (ms)**	**Versus best**
1.0	2,306	**2.3× worse**
1.1	1,954	1.9× worse
1.2	1,942	1.9× worse
1.5	1,006	best
2.0	1,053	1.0×
3.0	1,053	1.0×
4.0 (default)	**1,068**	1.06×

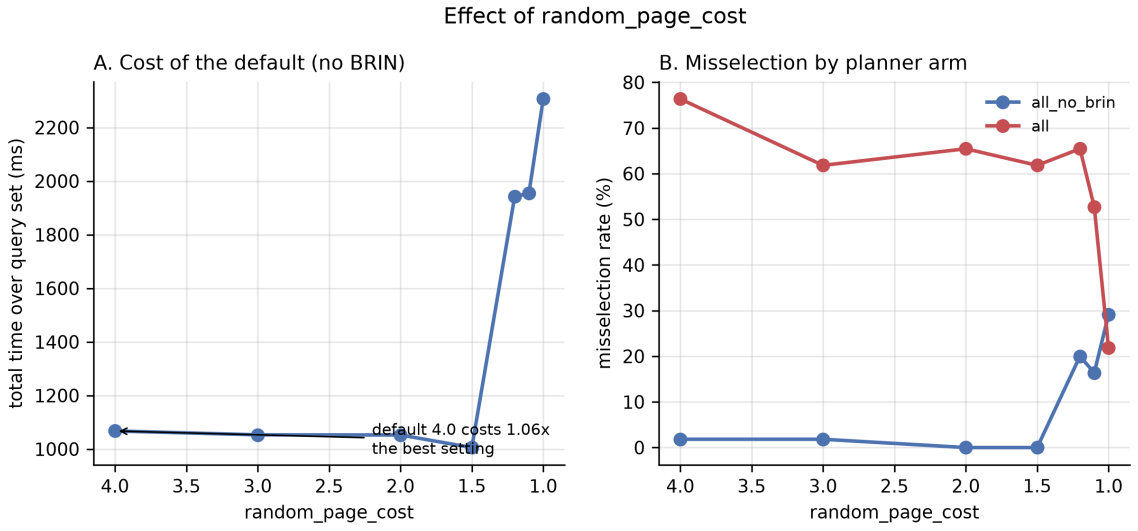

Figure 4: Effect of `random_page_cost`. Panel A, total query-set time; reducing the setting below 1.5 degrades the workload. Panel B, misselection rate by index configuration.

The access-path counts explain the mechanism: at `random_page_cost` $= 1.0$ the optimiser abandons bitmap scans entirely in favour of plain index scans, 136 index scans against zero bitmap scans, forfeiting the physical-order sorting that makes bitmap heap access efficient.

A single-query measurement early in this study suggested the default cost 24×; that observation was accurate but unrepresentative, and is retained in the artifact as a documented instance of how anecdotal tuning evidence misleads.

4.5 RQ2 and RQ4c: A Co-located BRIN Index

BRIN indexes are small and cheap to build, so adding one is commonly regarded as harmless. It is not, while the table is buffer-pool resident (Table 11, Figure 5).

Table 11: Effect of adding a BRIN index alongside an existing B-tree on the same column.

Scale	With BRIN	Without BRIN	Median R	Max R
10^6	**73.3%**	2.3%	2.06 vs 1.01	**18.78**
10^7	30.1%	32.3%	1.04 vs 1.04	2.63

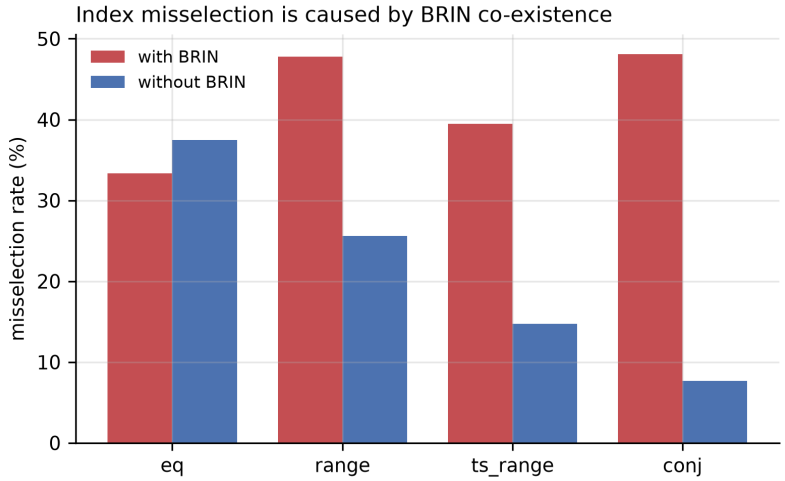

Figure 5: Misselection rate by query family with and without BRIN in the index set, 10^6 rows. The conjunctive family is the control: no BRIN index covers those columns and correspondingly no effect appears.

In controlled diagnostics with statistics held byte-identical, individual queries degraded by up to **194 times**, from 0.34 ms to 65.95 ms, purely because a BRIN index existed alongside the B-tree. The conjunctive family serves as an internal control: no BRIN index covers columns a or b, and correspondingly the matched comparison shows a median speedup of $0.90\times$ from removing BRIN there, against $2.83\times$ and $2.78\times$ for the range families. The effect appears exactly where the mechanism predicts.

Root cause, confirmed in source. With both indexes present and a bitmap plan forced, the optimiser selected BRIN in 16 of 16 cases, and in ten of those the B-tree bitmap path was cheaper *by the optimiser's own cost model*, once by a factor of nine. A cost-based optimiser does not knowingly select a path it prices as nine times more expensive, so that path cannot be reaching the final comparison.

Inspection of the PostgreSQL 17 source identifies the responsible line. Bitmap path selection is performed by `choose_bitmap_and()` in `src/backend/optimizer/path/indxpath.c`. Before comparing costs it deduplicates candidates, described in the source comment as detecting "any paths that use exactly the same set of clauses" and keeping "only the cheapest-to-scan of any such groups". A BRIN and a B-tree index on the same column serving the same predicate use precisely the same clause set, so only one survives. The survivor is chosen by `cost_bitmap_tree_node()`, which returns the index path's `indextotalcost`, that is the cost of the *index-side* scan alone, ignoring the heap access that consumes the resulting bitmap (Table 12).

Table 12: The deduplication tiebreak. BRIN wins on the only metric examined, and loses by 194× on the metric that is not.

	Index-side cost (examined)	**Actual query time (ignored)**
BRIN	**12.13** (wins tiebreak)	65.95 ms
B-tree	213.35 (discarded)	**0.34 ms**

This makes the failure systematic rather than incidental. A very small index is BRIN's design goal, so its index-side cost is necessarily far below a B-tree's; BRIN therefore wins the tiebreak essentially by construction. Yet the entire cost difference lies on the heap side, which the tiebreak never examines: the B-tree bitmap is exact while the BRIN bitmap is lossy at block granularity, forcing a recheck of 990,000 rows. Figure 6 shows why BRIN depends so completely on physical clustering.

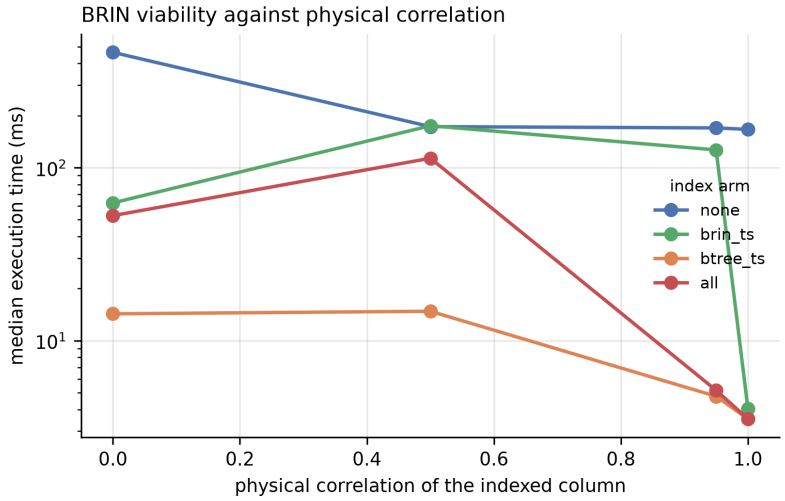

Figure 6: Execution time against physical correlation of the indexed column, by index configuration. BRIN is competitive only as correlation approaches unity.

4.6 RQ5: Locating the Boundaries

Comparing 10^6 against 10^7 rows shows the penalty present at the smaller scale and absent at the larger. Two points, however, cannot distinguish gradual decay from a sharp transition, nor reveal non-monotonic behaviour between them. Five intermediate scales were therefore measured (Table 13, Figure 7).

Table 13: The BRIN penalty across table size. Frequency and severity collapse at different scales.

Million	**Heap**	**Blocks read**	**Misselection %**		**Max regret**	
rows	**(MB)**	**outside pool**	with BRIN	without	with BRIN	without
1.00	111.8	0	**71.4**	0.0	18.78	1.06
1.25	139.8	0	37.9	34.5	**38.81**	1.67
1.50	167.8	0	31.0	17.2	**40.32**	1.76
2.00	223.2	0	25.8	12.9	**38.34**	1.77
3.00	335.2	0	30.3	12.1	**41.51**	1.67
5.00	558.2	1,902	36.4	30.3	6.98	1.81
10.00	1,116.2	89,081	38.2	44.4	1.92	3.70

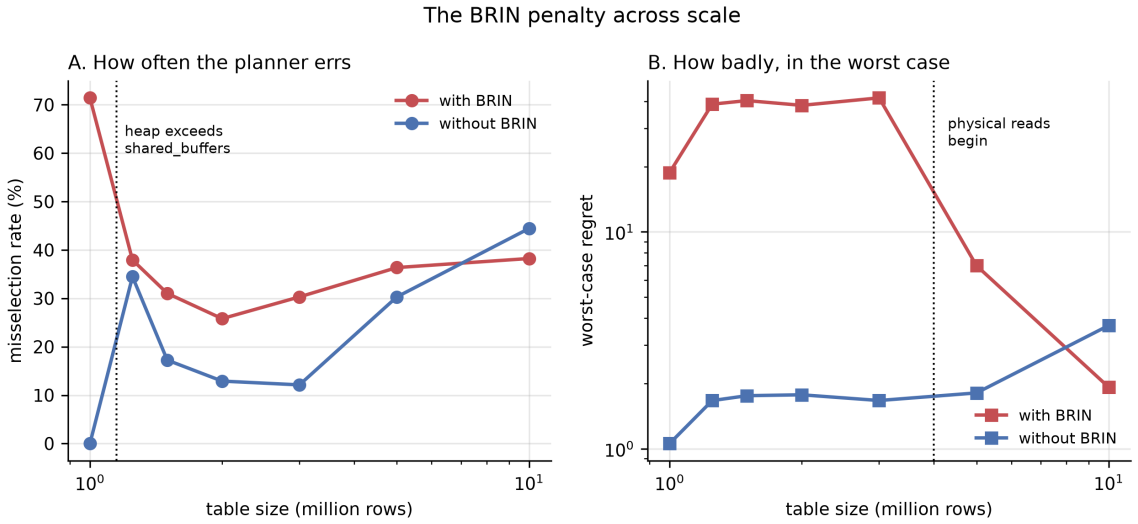

Figure 7: The BRIN penalty across scale. Panel A, how often the optimiser errs. Panel B, worst-case regret. The two collapse at different table sizes, near different system boundaries.

There are two transitions, not one. **Frequency collapses at the buffer pool boundary**: the gap falls from 71.4 percentage points at 10^6 rows to 3.4 at 1.25×10^6, precisely where the heap crosses `shared_buffers`, 111.8 MB against a 128 MB pool becoming 139.8 MB. **Severity collapses later, after first intensifying**: worst-case regret roughly doubles at that same boundary, from 18.78 to 38.81, remains near 40 through 3×10^6, and falls only at 5×10^6 and 10^7. That later collapse coincides with the onset of physical reads, which are zero up to three million rows.

The practical implication inverts the naive reading. A deployment whose table is two to three times `shared_buffers` experiences the most severe individual regressions measured anywhere in this study, worse than at the scale where the effect was first identified. Reporting only the two-point comparison would have concluded that the penalty diminishes with scale, implicitly reassuring exactly the practitioners most at risk.

4.7 RQ6: Which Findings Generalise?

A single-system study cannot separate properties of cost-based access-path selection from artefacts of one implementation. The 10^6-row sweep was therefore repeated on MariaDB 10.4.28 using the same generated data, the same queries and the same metric, with `innodb_buffer_pool_size` matched to PostgreSQL's `shared_buffers`. InnoDB offers neither BRIN nor hash indexes, so the comparison is restricted to the six configurations both systems support and the BRIN result cannot be replicated.

Selection quality does not generalise. On identical data the two optimisers differ sharply (Table 14). MariaDB errs far more often but far more mildly.

Table 14: Same data, same queries, same metric, 10^6 rows, arms common to both systems.

System	Queries	Misselection	Median R	p90 R	Max R
PostgreSQL 17.1	130	2.3%	1.013	1.13	1.39
MariaDB 10.4.28	153	66.0%	1.341	2.43	7.91

The chosen plans explain the difference. PostgreSQL selects bitmap scans almost everywhere, converting random heap access into ordered access. MariaDB has no equivalent intermediate for a single-index predicate: it either performs a `ref` lookup or scans the table, and it fell back to a full scan for 66 queries where PostgreSQL used none.

The correlated-predicate failure does generalise. MariaDB estimates conjunctive predicates *exactly* at dependency strengths 0.25 through 0.75, precisely where PostgreSQL is wrong by factors of 26 to 82 (Table 15, Figure 8). It achieves this through index dives: with both key parts of the composite index bound it descends the B-tree and counts, measuring where PostgreSQL estimates.

Table 15: Median q-error on dependent conjunctive predicates. MariaDB is exact until it changes plan shape.

Dependency strength	PostgreSQL	MariaDB	MariaDB plan
0.25	26.39	**1.00**	`ref` on composite index
0.50	49.01	**1.00**	`ref` on composite index
0.75	81.82	**1.00**	`ref` on composite index
1.00	98.65	**55.84**	`index_merge`

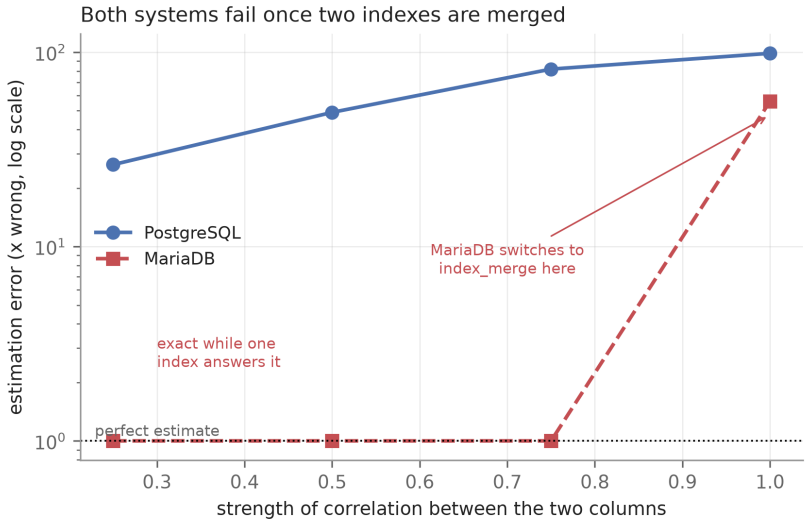

Figure 8: Estimation error on dependent conjunctive predicates. MariaDB is exact, and therefore flat, while a single composite index answers the predicate. Its error appears the moment it switches plan.

It is tempting to conclude that the merge plan shape is the culprit, and this study initially did. A direct test refutes it: disabling `index_merge` entirely forces a `ref` scan with a residual filter, changing the plan shape completely, and the estimation error is unchanged, 55.59 against 55.84. The merge is where the error becomes visible, not where it originates. The correct attribution is that both systems estimate a conjunction accurately only when a single index covers both columns; once none does, both combine per-column selectivities under independence and both fail, whatever operator performs the combination.

The two systems degrade in opposite directions. Measuring both across the full scale range shows they do not merely differ in degree (Table 16, Figure 9). PostgreSQL's misselection rate rises with scale while its worst case stays bounded below $3.7\times$; MariaDB's rate falls while its worst case grows monotonically to $30.85\times$.

Table 16: Both systems across scale. The oracle for each is restricted to configurations both provide, so neither is credited with a plan the other could not form.

Million	**PostgreSQL**		**MariaDB**	
rows	Misselection	Max R	Misselection	Max R
1.00	0.0%	1.05	52.9%	3.33
1.25	27.6%	1.67	74.3%	6.52
1.50	6.9%	1.76	77.8%	10.85
2.00	6.5%	1.36	75.7%	15.21
3.00	6.1%	1.67	36.1%	18.77
5.00	12.1%	1.80	37.9%	27.78
10.00	**36.1%**	**3.70**	**12.4%**	**30.85**

The plan repertoire explains it. PostgreSQL's bitmap intermediate bounds the cost of a

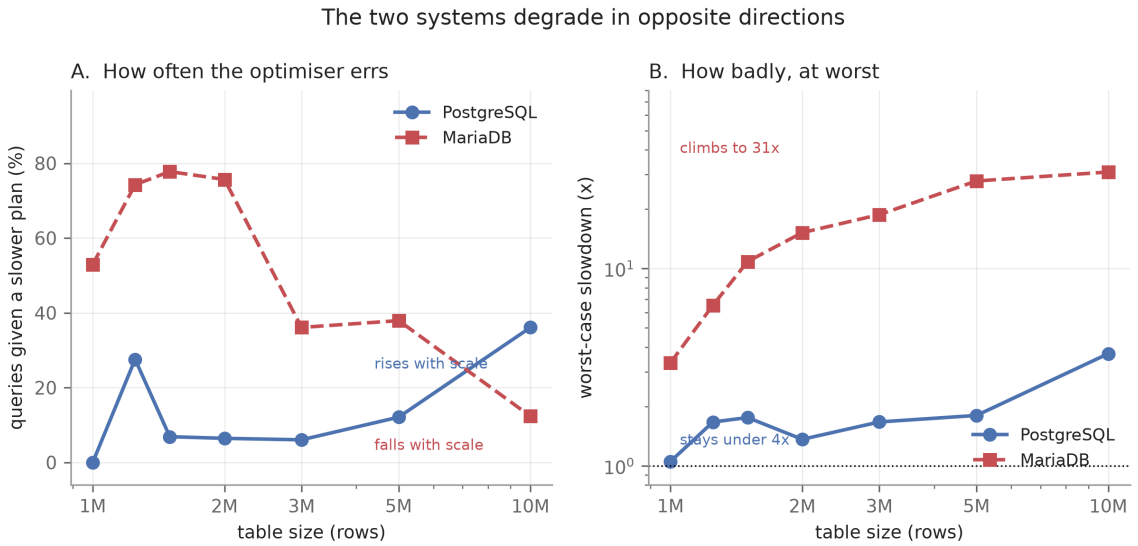

Figure 9: Frequency and severity plotted on separate panels rather than a shared axis, since they carry different units. Panel B is logarithmic.

mistake, since a wrong choice still reads the heap in physical order. MariaDB has no such ceiling: a mistaken index scan degenerates into random single-row fetches whose cost grows with the table. This cautions against summarising optimiser quality with a single rate. An optimiser wrong 36% of the time but never by more than $3.7\times$ is preferable, for most workloads, to one wrong 12% of the time but occasionally by $31\times$.

4.8 Computational Cost

The two systems also differ in where they spend time (Table 17, Figure 10). This is measured work, every repetition of every query, not wall clock.

Table 17: Total measured work at 10^7 rows, restricted to the index configurations both systems provide.

System	Query execution	Index construction	Records
PostgreSQL 17.1	50.8 min	45.6 min	1,232
MariaDB 10.4.28	781.8 min	19.3 min	1,232
Ratio	**15.4× slower**	**2.4× faster**	

Two observations follow. First, regret and total cost measure different things: MariaDB's median regret of 1.34 implies mild inefficiency, yet its execution cost is $15.4\times$ PostgreSQL's, because it usually chooses close to the best plan *available to it* and the plans available to it are worse. Second, the profiles invert. PostgreSQL divides its time roughly evenly between building and querying; MariaDB builds in 19 minutes then queries for thirteen hours. A study measuring only build cost, or only query cost, would rank the two systems in opposite orders,

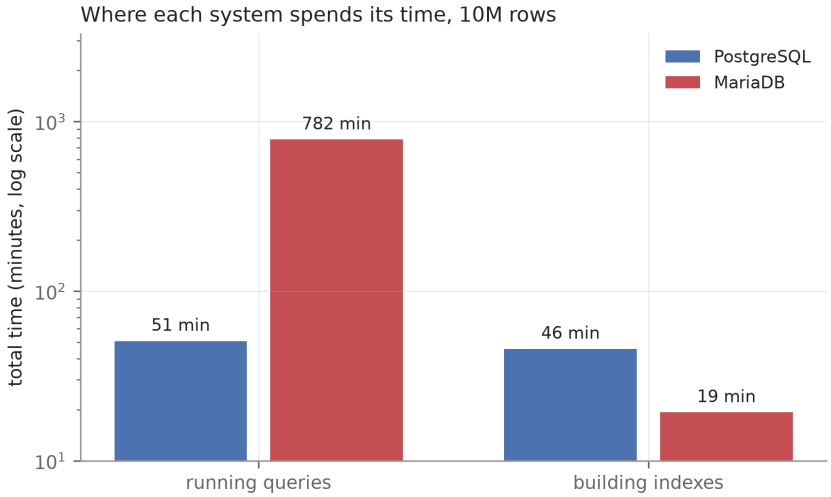

Figure 10: Total measured work at 10^7 rows. Note the logarithmic axis.

which cautions index-selection work that models index benefit while treating construction as negligible [27].

4.9 Incidental Finding: Hash Index Construction Under Skew

Hash index build time proved acutely sensitive to value skew (Table 18). Zipfian values collide into few hash buckets, producing long overflow chains, and PostgreSQL does not parallelise hash construction as it does B-tree construction. Hash index creation cannot be assumed cheap on skewed columns.

Table 18: Index construction time at 10^7 rows by value skew.

Dataset	`zipf_s`	Hash build	B-tree build
baseline	0.0	11.6 s	3.2 s
skew05	0.5	16.7 s	2.3 s
skew10	1.0	187.7 s	2.5 s
skew15	**1.5**	**2,152.9 s**	2.3 s

4.10 Where the Errors Sit

Figure 11 places the results in context. Misselection concentrates between roughly one and ten percent selectivity, where the cost of index access and of a sequential scan converge and a small pricing error therefore suffices to flip the decision. Outside that band the correct choice is obvious enough that the optimiser makes it.

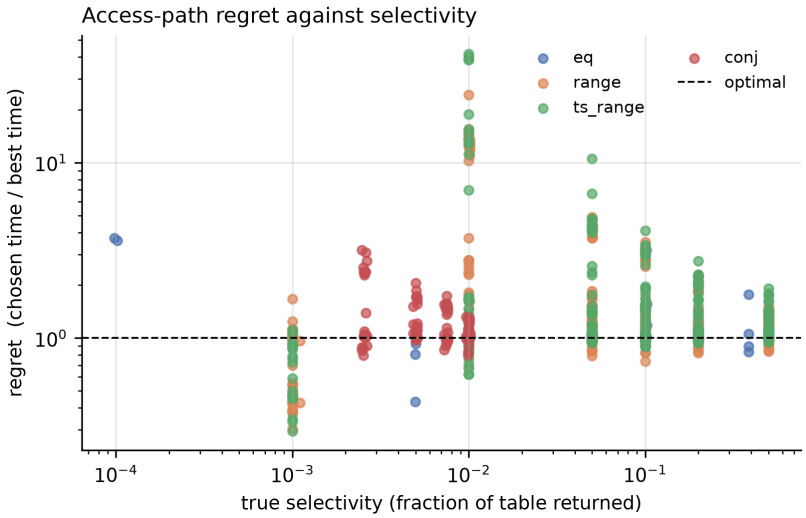

Figure 11: Access-path regret against true selectivity, by query family. Both axes logarithmic; the dashed line marks an optimal choice.

5 Discussion

5.1 Key Insights

The three practices fail for a shared reason: each optimises a single quantity, while plan quality depends on several.

Extended statistics optimise **estimate accuracy**. A more accurate estimate improves the plan only if applied consistently wherever the plan is priced, and Section 4.3 shows it is not. The partial correction is worse than none, because it makes two paths priced under different assumptions directly comparable when they are not.

Reducing `random_page_cost` optimises for **storage latency**. The premise is sound, but the same constant governs the choice between bitmap and plain index access, so reducing it improves one decision while degrading another.

Adding a BRIN index optimises for **index size and construction cost**, on both of which BRIN genuinely excels. But index availability also determines which paths the optimiser generates, and a cheap index can capture the bitmap path and exclude a superior one.

In each case the advice is correct about the quantity it names and wrong about the outcome. This generalises: mechanism-based tuning advice is systematically vulnerable whenever the mechanism is real but not the only mechanism in play, and the corrective is workload-level measurement.

5.2 Connection to Course Concepts

The work applies several core topics from the course directly. *Indexing*: the trade-off between B-tree, hash and BRIN structures is measured rather than asserted, including BRIN's depen-

dence on physical clustering and hash construction's dependence on value distribution. *Query processing and optimisation*: the System R cost-based framework [2] is exercised at its weakest point, the transition between access paths, and the propagation of selectivity estimates through a plan tree is observed directly in the listing of Section 4.3. *Storage management*: the buffer-pool residency boundary of Section 4.6 is exactly the distinction between logical reads served from `shared_buffers` and physical reads issued beyond it, and it determines whether an entire finding holds. *Database architecture comparison*: the cross-system results show how a single architectural choice, the presence or absence of a bitmap intermediate, propagates into measurable differences in both plan quality and total cost.

5.3 Recommendations for Practitioners

1. Do not create extended statistics reflexively. Verify that the *plan* improved, not merely the estimate; the two are separable and can move in opposite directions.
2. Do not reduce `random_page_cost` below approximately 1.5 without workload measurement. The default was within 6% of optimal here.
3. Treat a BRIN index co-located with a B-tree on the same column as a risk requiring measurement, with particular attention when the table is between one and three times `shared_buffers`.
4. Do not assume hash index creation is inexpensive on skewed columns.

5.4 Limitations and Threats to Validity

Internal validity. Regret is a lower bound, since the denominator is the fastest measured configuration rather than a proven optimum. Values marginally below 1.0 occur because the free-choice configuration can combine indexes in ways no single restricted configuration can, and are reported rather than clamped. The misselection threshold of 1.2 is a judgement call; full distributions are published so alternatives may be applied.

Construct validity. The path-elimination mechanism of Section 4.5 is confirmed against the PostgreSQL 17 source, but that reading is static: the planner was not instrumented to observe the discarded path directly, so the identification rests on the code path being the only one consistent with the observed costs. The extended-statistics result requires a redundant index set, a composite index alongside both single-column indexes, which is common but not universal.

External validity. Two DBMSs on one hardware configuration, and the MariaDB arm is measured at fewer scales than PostgreSQL, so the boundaries of Section 4.6 are established for PostgreSQL only. Two systems suffice to show a behaviour is not universal, as with selection quality, but are suggestive rather than conclusive when both agree, since two optimisers may share an inherited design assumption. All measurements are warm-cache; reliably evicting the operating system page cache is not portable, so cold-storage behaviour is out of scope rather than approximated. Because the host has 23.8 GB of memory, the page cache retains the data

even at 10^7 rows, so the larger scales test departure from the *buffer pool* rather than from memory, and the two cannot be separated on this hardware. Only single-relation access paths are considered.

6 Conclusion and Recommendations

PostgreSQL 17's access-path selection is close to optimal under a conventional index set, selecting the best available plan for 97.7% of the queries measured. Where it fails, cardinality misestimation is not the cause: no query in the experiment exhibits both severe misestimation and material regret, in contrast to the established result for join ordering. That quality is an implementation property rather than a general one, since the same workload on MariaDB misselects for 66.0% of queries.

The failures trace instead to cost modelling and path generation, and are triggered by the three practices intended to prevent them: extended statistics, which improve estimates by up to 108× while slowing queries by up to 2.7×; reducing `random_page_cost`, which costs a factor of 2.3; and a co-located BRIN index, which raises misselection to 73.3% and can slow individual queries by 194×.

The boundaries of that last result proved as important as the effect, and measuring them changed what could be claimed. A two-point comparison would have supported the conclusion that the penalty vanishes with scale; five intermediate scales show its frequency collapses once the heap exceeds `shared_buffers` while its worst case doubles at that same point. The most severe regressions occur at two to three times the buffer pool size, not at the smallest scale.

A tuning recommendation is not correct or incorrect in isolation; it is correct or incorrect under conditions, and those conditions are measurable. Reporting an effect without its boundaries invites practitioners to apply it where it does not hold, or to dismiss it where it does.

Future work. The deduplication tiebreak suggests a concrete remedy worth evaluating: breaking ties on full scan cost rather than index-side cost, at the price of additional planning time. Extending the MariaDB arm across all scales would establish whether the buffer-pool boundaries are likewise implementation-specific, and a third optimiser would test whether the per-column composition weakness is genuinely architectural. Finally, hardware permitting controlled page-cache eviction would separate buffer-pool residency from operating system cache residency, a distinction this platform cannot make.

7 References

[1] I. Sajin, *Access-path selection benchmark for PostgreSQL: Code, raw measurements and analysis*, https://github.com/Imtiaj-Sajin/Reseach-on-Database-Architecture, Accessed 16 August 2026, 2026.

[2] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1979, pp. 23–34. DOI: 10.1145/582095.582099

[3] PostgreSQL Global Development Group, *PostgreSQL 17 documentation: CREATE STATISTICS*, https://www.postgresql.org/docs/17/sql-createstatistics.html, Accessed 16 August 2026, 2024.

[4] PostgreSQL Global Development Group, *PostgreSQL 17 documentation, section 19.7: Query planning configuration*, https://www.postgresql.org/docs/17/runtime-config-query.html, Accessed 16 August 2026, 2024.

[5] PostgreSQL Global Development Group, *PostgreSQL 17 documentation, chapter 68: BRIN indexes*, https://www.postgresql.org/docs/17/brin.html, Accessed 16 August 2026, 2024.

[6] G. Graefe, "The five-minute rule 20 years later, and how flash memory changes the rules," *Communications of the ACM*, vol. 52, no. 7, pp. 48–59, 2009. DOI: 10.1145/1538788.1538805

[7] S.-W. Lee, B. Moon, and C. Park, "Advances in flash memory SSD technology for enterprise database applications," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2009, pp. 863–870. DOI: 10.1145/1559845.1559937

[8] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015. DOI: 10.14778/2850583.2850594

[9] V. Leis et al., "Query optimization through the looking glass, and what we found running the join order benchmark," *The VLDB Journal*, vol. 27, no. 5, pp. 643–668, 2018. DOI: 10.1007/s00778-017-0480-7

[10] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proc. ACM Symp. on Principles of Database Systems (PODS)*, 1998, pp. 34–43. DOI: 10.1145/275487.275492

[11] G. Graefe, "Query evaluation techniques for large databases," *ACM Computing Surveys*, vol. 25, no. 2, pp. 73–169, 1993. DOI: 10.1145/152610.152611

[12] PostgreSQL Global Development Group, *PostgreSQL 17 documentation, section 63.6: Index cost estimation functions*, https://www.postgresql.org/docs/17/index-cost-estimation.html, Accessed 16 August 2026, 2024.

[13] D. Comer, "The ubiquitous b-tree," *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, 1979. DOI: 10.1145/356770.356776

[14] G. Graefe, "Modern b-tree techniques," *Foundations and Trends in Databases*, vol. 3, no. 4, pp. 203–402, 2011. DOI: 10.1561/1900000028

[15] M. Stonebraker and U. Cetintemel, "One size fits all: An idea whose time has come and gone," in *Proc. 21st Int. Conf. on Data Engineering (ICDE)*, 2005, pp. 2–11. DOI: 10.1109/ICDE.2005.1

[16] PostgreSQL Global Development Group, *PostgreSQL 17 documentation, section 14.2: Statistics used by the planner*, https://www.postgresql.org/docs/17/planner-stats.html, Accessed 16 August 2026, 2024.

[17] W. Effelsberg and T. Haerder, "Principles of database buffer management," *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 560–595, 1984. DOI: 10.1145/1994.2022

[18] Y. E. Ioannidis and S. Christodoulakis, "On the propagation of errors in the size of join results," *ACM SIGMOD Record*, vol. 20, no. 2, pp. 268–277, 1991. DOI: 10.1145/119995.115835

[19] G. Moerkotte, T. Neumann, and G. Steidl, "Preventing bad plans by bounding the impact of cardinality estimation errors," *Proc. VLDB Endowment*, vol. 2, no. 1, pp. 982–993, 2009. DOI: 10.14778/1687627.1687738

[20] Y. Han et al., "Cardinality estimation in DBMS: A comprehensive benchmark evaluation," *Proc. VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2021. DOI: 10.14778/3503585.3503586

[21] P. Negi et al., "Flow-loss: Learning cardinality estimates that matter," *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2019–2032, 2021. DOI: 10.14778/3476249.3476259

[22] W. Wu, Y. Chi, S. Zhu, J. Tatemura, H. Hacigumus, and J. F. Naughton, "Predicting query execution time: Are optimizer cost models really unusable?" In *Proc. IEEE 29th Int. Conf. on Data Engineering (ICDE)*, 2013, pp. 1081–1092. DOI: 10.1109/ICDE.2013.6544899

[23] R. Marcus et al., "Neo: A learned query optimizer," *Proc. VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019. DOI: 10.14778/3342263.3342644

[24] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2021, pp. 1275–1288. DOI: 10.1145/3448016.3452838

[25] H. Lan, Z. Bao, and Y. Peng, "A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration," *Data Science and Engineering*, vol. 6, no. 1, pp. 86–101, 2021. DOI: 10.1007/s41019-020-00149-7

[26] S. Chaudhuri and V. R. Narasayya, "An efficient cost-driven index selection tool for Microsoft SQL Server," in *Proc. 23rd Int. Conf. on Very Large Data Bases (VLDB)*, 1997, pp. 146–155.

[27] J. Kossmann, S. Halfpap, M. Jankrift, and R. Schlosser, "Magic mirror in my hand, which is the best in the land? an experimental evaluation of index selection algorithms," *Proc. VLDB Endowment*, vol. 13, no. 11, pp. 2382–2395, 2020. DOI: 10.14778/3407790.3407832

[28] C. Wongkham, B. Lu, C. Liu, Z. Zhong, E. Lo, and T. Wang, "Are updatable learned indexes ready?" *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 3004–3017, 2022. DOI: 10.14778/3551793.3551848

[29] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2018, pp. 489–504. DOI: 10.1145/3183713.3196909

[30] M. Raasveldt, P. Holanda, T. Gubner, and H. Muehleisen, "Fair benchmarking considered difficult: Common pitfalls in database performance testing," in *Proc. 7th Int. Workshop on Testing Database Systems (DBTest)*, 2018, pp. 1–6. DOI: 10.1145/3209950.3209955

[31] P. Boncz, T. Neumann, and O. Erling, "TPC-H analyzed: Hidden messages and lessons learned from an influential benchmark," in *Performance Characterization and Benchmarking, LNCS 8391*, Springer, 2013, pp. 61–76. DOI: 10.1007/978-3-319-04936-6_5

A Query Execution Plans

All plans below are unmodified `EXPLAIN` output captured during the study; each can be regenerated from the artifact [1].

A.1 The BRIN Effect With Statistics Held Constant

10^6 rows, `physical_corr = 0.5`, both indexes on `ts`. `ANALYZE` was run once and never again, so both plans come from byte-identical statistics. The generator's realised rank correlation was 0.4984 and PostgreSQL's own `pg_stats` estimate 0.49863.

```
Bitmap Heap Scan on t_brindiag  (cost=14.56..14858.18 rows=9705) (actual
   rows=10000)
  Rows Removed by Index Recheck: 990000
  Heap Blocks: lossy=14304
  ->  Bitmap Index Scan on ix_brindiag_brin_ts  (cost=0.00..12.13 rows
     =35975)
Execution Time: 132.691 ms
```

Listing 2: Plan chosen with both indexes available.

```
Bitmap Heap Scan on t_brindiag  (cost=215.87..14045.79 rows=10092) (actual
   rows=10000)
  Heap Blocks: exact=4293
  ->  Bitmap Index Scan on ix_brindiag_btree_ts  (cost=0.00..213.35 rows
     =10092)
Execution Time: 3.416 ms
```

Listing 3: Same query, same statistics, BRIN index removed.

Note `lossy=14304` against `exact=4293`, and 132.691 ms against 3.416 ms. The optimiser priced the chosen plan at 14,858 and the unchosen one at 14,046, so the plan it did not use was the cheaper by its own model.

A.2 Path Suppression Across the Correlation Range

Bitmap plans forced by disabling other access methods, statistics held constant within each row. BRIN was selected in 16 of 16 cases.

The largest penalty observed is the 0.95-correlation, 0.1% selectivity case: 62.07 ms against 0.09 ms, a factor of 690.

A.3 Extended Statistics Change the Plan for the Worse

10^6 rows, `dep_strength = 1.0`, query `SELECT * FROM t_ext WHERE a = 57 AND b = 84`, true rows 10,216. Identical table, data, indexes and session.

Table 19: Forced bitmap plans with both indexes present.

Corr.	Query	BRIN cost	BRIN ms	B-tree cost	B-tree ms
0.0	0.1% range	29,320	65.95	3,166	**0.34**
0.0	1% range	29,322	66.73	13,857	5.14
0.5	0.1% range	15,338	66.33	3,174	**0.21**
0.5	1% range	14,852	63.90	13,848	2.76
0.95	0.1% range	13,511	62.07	3,294	**0.09**
0.95	1% range	15,365	66.45	14,201	0.76

```
Bitmap Heap Scan on t_ext  (cost=5.38..356.28 rows=93) (actual rows=10216)
  Heap Blocks: exact=7317
  ->  Bitmap Index Scan on ix_ext_ab  (cost=0.00..5.35 rows=93)
```

Listing 4: Without extended statistics. Median of 15 runs: 4.059 ms (stdev 0.473).

```
Bitmap Heap Scan on t_ext  (cost=219.84..559.77 rows=9467) (actual rows
   =10216)
  Heap Blocks: exact=7317
  ->  BitmapAnd  (cost=219.84..219.84 rows=90)
        ->  Bitmap Index Scan on ix_ext_b  (cost=0.00..107.43 rows=9467)
        ->  Bitmap Index Scan on ix_ext_a  (cost=0.00..107.43 rows=9467)
```

Listing 5: With extended statistics. Median of 15 runs: 5.171 ms (stdev 0.261).

The two timing distributions do not overlap, so this is not measurement noise. Both plans fetch the identical 7,317 heap blocks.

A.4 Buffer Pool Residency

`shared_read` counts blocks fetched from outside the buffer pool. Zero at one million rows confirms full residency; 89,081 at ten million confirms the working set has left the pool. This is the independent evidence that the two scales are genuinely different regimes.

Table 20: Buffer statistics by scale.

Scale	shared_hit	shared_read
1,000,000 rows	13,194	**0**
10,000,000 rows	11,314	**89,081**

B Reproducibility

All code, raw measurements, figures and an executable analysis notebook are published [1]. No dataset is distributed: all data is generated from a fixed seed, so the corpus is byte-identical on

any machine.

```
pip install -r research/requirements.txt
psql -c "CREATE TABLESPACE ts_dtms LOCATION '/path/with/space';"

python research/tests/test_generator.py         # validate the generator
python research/tests/test_range_targeting.py   # validate query
    construction
python research/run_experiment.py --rows 1000000
python research/run_experiment.py --rows 10000000
python research/run_rpc_sweep.py
python research/run_experiment_mysql.py --rows 1000000
python research/analyze.py                      # regenerates every figure
```

Listing 6: Complete reproduction sequence.

Runs are resumable at the granularity of a single measurement: every result is appended to disk as it is taken, and the resume key is (dataset, index configuration, query, extended-statistics state). This was exercised repeatedly during the study, by two power interruptions and by three MariaDB crashes, and in each case the loss was a single in-flight query. Each record retains all individual repetitions rather than only the median, so the variance claims in this report can be checked independently.

C Individual Contribution Statement

Table 21: Individual contributions.

Member	Contribution
Md. Imtiaj Alam Sajin	Experimental design; benchmark implementation; data generation and validation; measurement infrastructure for both database systems; source-level diagnosis; analysis and figure generation; all sections of this report.